

MODULE 04

# Modern .NET API and Data Access

Day 1 · Core MVP Module

## Modern .NET API and Data Access



Client Request

HTTP GET



Minimal API

EF Core query



Azure SQL

# LEARNING OBJECTIVES

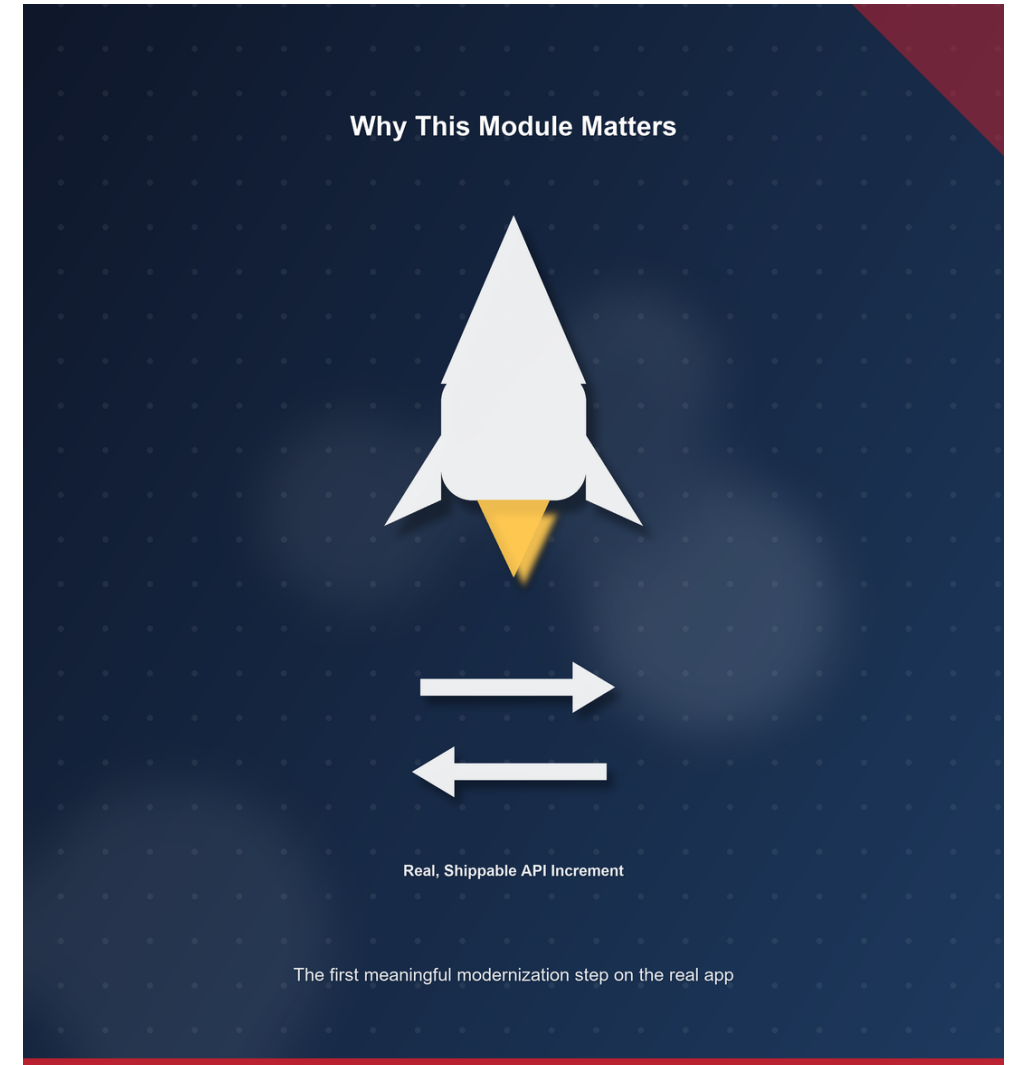
- Build a modern API endpoint using current .NET patterns.
- Integrate data access through EF Core or Dapper.
- Implement robust validation and configuration handling.



# WHY THIS MODULE MATTERS

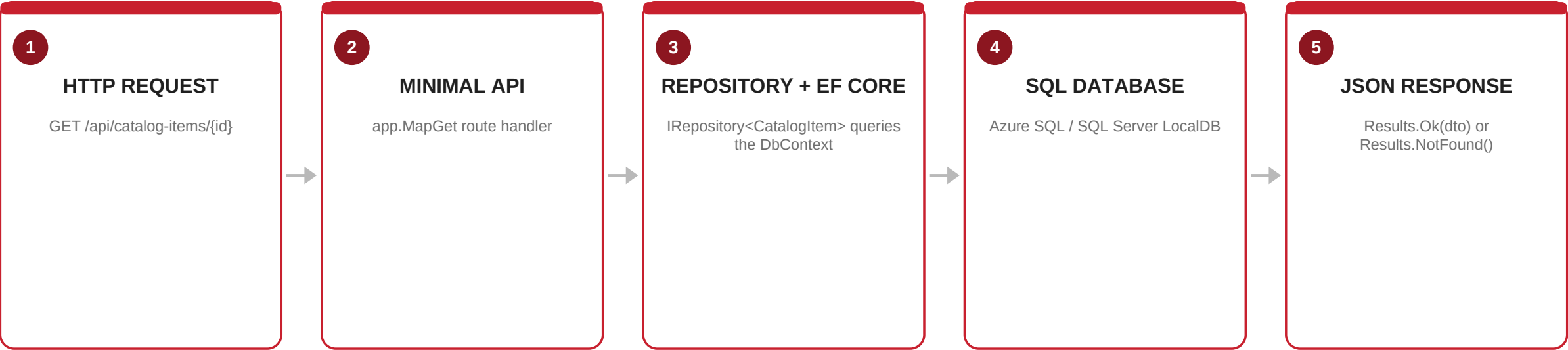
This module creates the first meaningful modernization increment.

It delivers a production-relevant API capability with data persistence and test coverage.



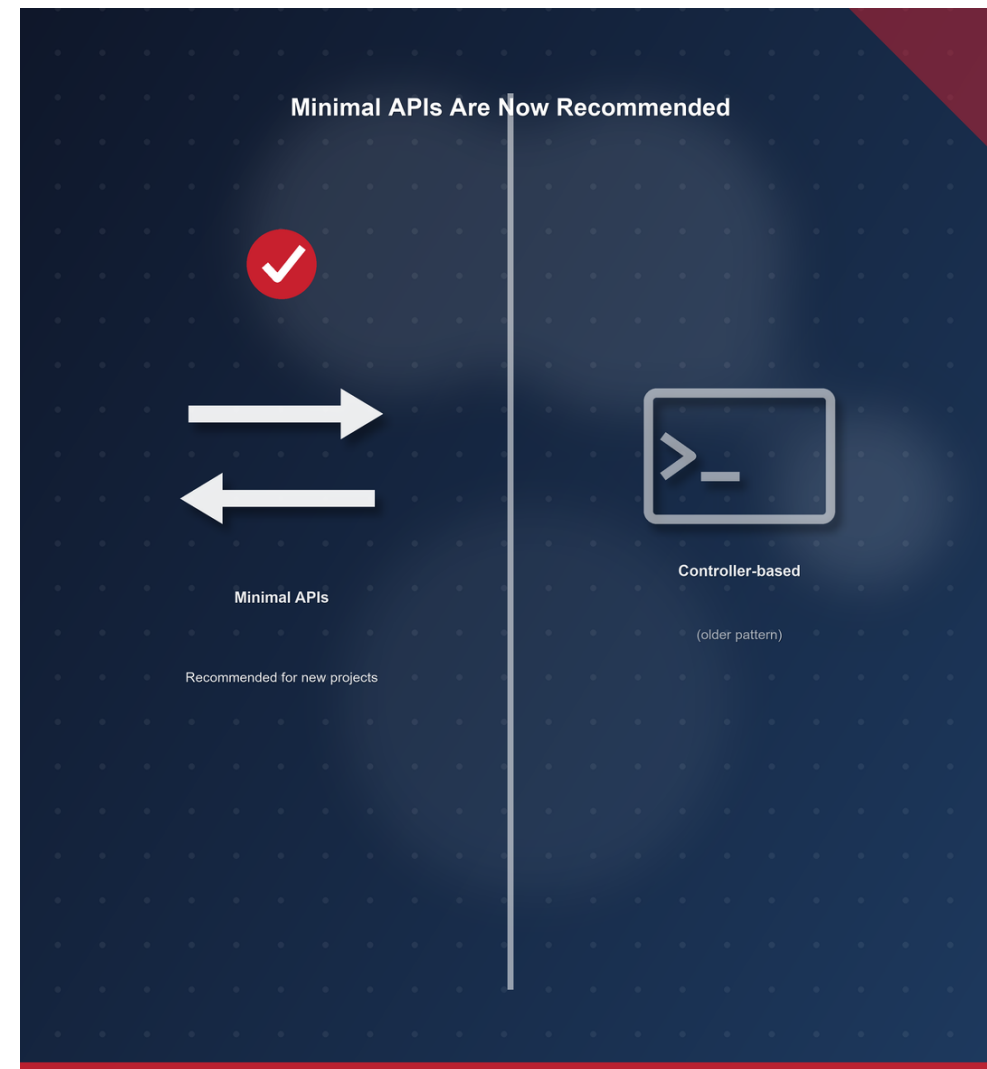
# CONCEPT MAP

The exact request path a Minimal API endpoint follows in eShopOnWeb's PublicApi project.



# FROM THE DOCS: WHY MINIMAL APIS

- Microsoft recommends Minimal APIs over controller-based APIs for new ASP.NET Core projects.
- Minimal APIs build working REST endpoints with minimal code — no controller classes or scaffolding.
- Endpoints are defined with `app.MapGet()`, `app.MapPost()`, `app.MapPut()`, `app.MapDelete()` on the `WebApplication` instance.
- Route handlers can request services directly — ASP.NET Core resolves them from DI automatically.
- Dapper is a lightweight object mapper for raw SQL; EF Core is a full ORM with change tracking and migrations.



# COURSEWARE FOCUS

- Minimal APIs
- Dependency injection
- Configuration
- Async/await
- Data access



# REAL ENDPOINT — ESHOPONWEB PUBLICAPI

REAL CODE — src/PublicApi/CatalogItemEndpoints/CatalogItemGetByIdEndpoint.cs

```
public class CatalogItemGetByIdEndpoint
    : IEndpoint<IResult, GetByIdCatalogItemRequest, IRepository<CatalogItem>>
{
    public void AddRoute(IEndpointRouteBuilder app)
    {
        app.MapGet("api/catalog-items/{catalogItemId}",
            async (int catalogItemId, IRepository<CatalogItem> itemRepository) =>
            {
                return await HandleAsync(
                    new GetByIdCatalogItemRequest(catalogItemId), itemRepository);
            })
            .Produces<GetByIdCatalogItemResponse>()
            .WithTags("CatalogItemEndpoints");
    }

    // HandleAsync queries the repository, then returns
    // Results.NotFound() or Results.Ok(response)
}
```

*This is the exact Minimal API + repository pattern Lab 04 extends with a new endpoint.*

# TOOLS & RESOURCES

---

## Sample Code

**.NET 8 / .NET 9 Web API Reference Samples (dotnet/samples)**

*Template for Minimal APIs, Dependency Injection, and Async/Await.*

## ORM / Database

**Entity Framework Core & Dapper docs and code snippets**

*Adding REST endpoints backed by Azure SQL / SQL Server LocalDB.*

## Security Guides

**Azure Key Vault references for ASP.NET Core**

*Secrets management lab exercise.*



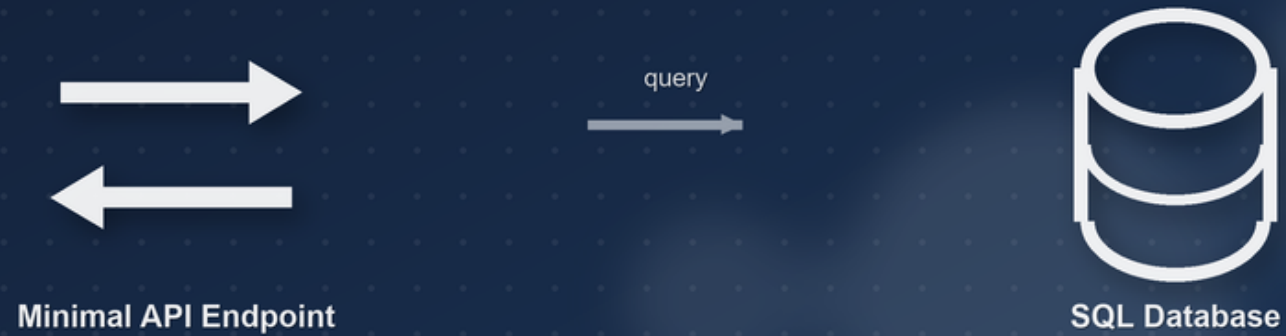
# SUPPORTING ASSETS

- App baseline: [course/repos/eShopOnWeb](#)
- Pattern references: [course/repos/samples](#)



## LAB 04: MODERN .NET API + SQL

### Lab 04: Modern .NET API + SQL



# LAB STEPS

Goal: Add a SQL-backed API endpoint and validate it end to end.

Tools: .NET SDK, SQL tooling · Cloud: optional Azure SQL, otherwise local SQL.

- Step 1 — Add a new API endpoint for a domain entity.
- Step 2 — Register services via dependency injection.
- Step 3 — Add EF Core or Dapper data access logic.
- Step 4 — Add configuration and secure secret handling.
- Step 5 — Add and run integration tests.
- Validation: the endpoint returns expected codes/schema, and integration tests pass.



## KNOWLEDGE CHECK: Modern .NET API and Data Access

---

Which two data-access approaches are named for connecting a modern .NET API to a database in this module?

- A. EF Core and Dapper
- B. jQuery and AJAX
- C. XML and SOAP
- D. FTP and SCP



EF Core

+



Dapper

## KNOWLEDGE CHECK: Modern .NET API and Data Access

Which two data-access approaches are named for connecting a modern .NET API to a database in this module?

A. EF Core and Dapper

B. jQuery and AJAX

C. XML and SOAP

D. FTP and SCP



EF Core

+



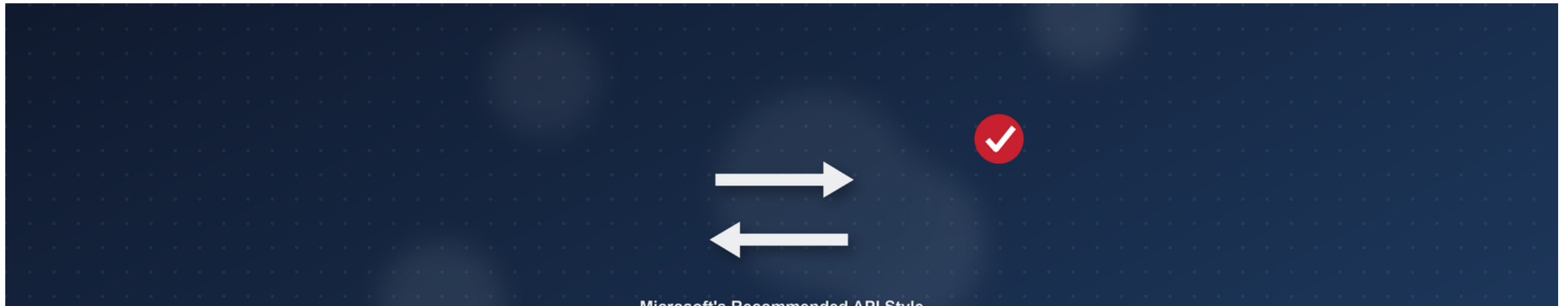
Dapper

## KNOWLEDGE CHECK: Modern .NET API and Data Access

---

Per Microsoft's own ASP.NET Core docs, which API style is now recommended for new projects?

- A. SOAP services
- B. Minimal APIs
- C. WCF
- D. Controller-based APIs only



## KNOWLEDGE CHECK: Modern .NET API and Data Access

---

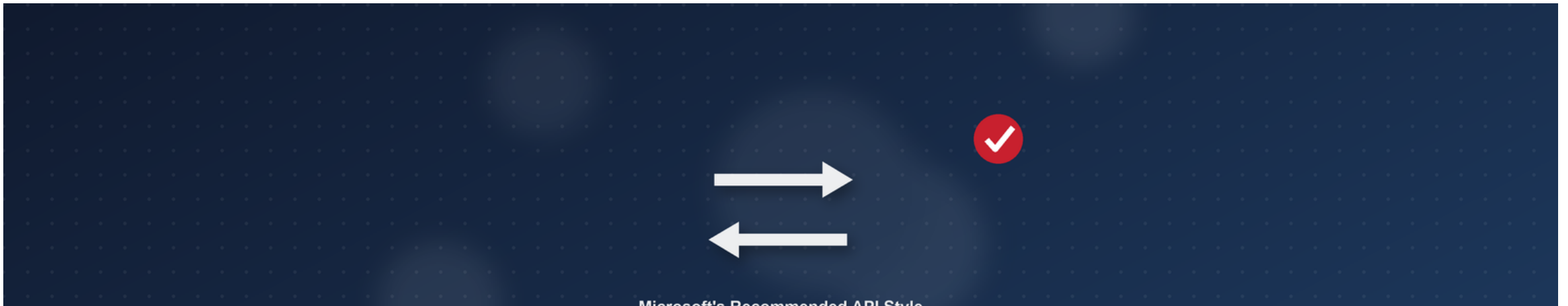
Per Microsoft's own ASP.NET Core docs, which API style is now recommended for new projects?

A. SOAP services

**B. Minimal APIs**

C. WCF

D. Controller-based APIs only



# SOURCES & FURTHER READING

---

## EXTERNAL DOCUMENTATION

### ■ APIs overview — ASP.NET Core

<https://learn.microsoft.com/en-us/aspnet/core/fundamentals/apis>

## REFERENCED IN THIS REPOSITORY

- `course/repos/eShopOnWeb/src/PublicApi/CatalogItemEndpoints/CatalogItemGetByIdEndpoint.cs`
- `course/mvp-delivery/labs/lab-04-modern-dotnet-api-sql.md`



# RECAP & SUCCESS CRITERIA

- Endpoint passes functional and integration checks and follows secure config handling.
- Tier: Core MVP Module

